

dInference: A Decentralized Protocol for Verifiable Large Language Model Inference on Solana

Permissionless GPU supply, optimistic challenge resolution, and constant-size activation-fingerprint verification

October 2026

Contents

Abstract	2
1. Motivation	2
1.1 Two problems, one network	2
1.2 Why this is a hard problem	3
1.3 The dInference approach	3
2. Protocol overview	4
3. Optimistic rollup rationale	4
4. The verification protocol	5
4.1 Fingerprint construction	5
4.2 Verifier recomputation	6
4.3 Why the activation level	6
4.4 Why not zkML	6
4.5 Measured timing	6
4.6 Improvements still open	7
5. Provider competition and supply-side dynamics	7
5.1 Current daemon: FCFS without filtering	8
6. Sybil resistance	8
7. On-chain program	9
7.1 Account model	9
7.2 Instructions	10
7.3 Committee selection	10
7.4 Early-finalize decidability	10
7.5 Stake invariants	10
8. Off-chain components	11

8.1 Fingerprint pipelines	11
8.2 Daemons	11
8.3 User SDK	11
9. Storage layer	11
10. Attack surface	12
11. Offline evaluation across model families	12
12. Limitations	12
13. References and reproducibility	13
Author’s note	13

Abstract

dInference is a **decentralized physical infrastructure network (DePIN)** for LLM inference: anyone with GPU capacity can register as a provider, serve user bounties, and get paid in SOL. Correctness is enforced by an **optimistic rollup** — the user who posted a bounty may dispute the response within a configurable window, and disputes trigger a verification round by a randomly-selected committee drawn from a per-model verifier pool. Verification is powered by an **activation fingerprint**: a seeded random projection of the final-layer hidden states produced during generation. The provider commits a 64-dimensional fingerprint on chain; verifiers independently recompute it from the response under the claimed model.

The check is a heuristic rather than a cryptographic proof, but empirically very accurate. In a cross-family evaluation of 500 prompts $\times$ 3 scenarios (1,500 samples), it separates honest and cheating runs with **0% false-positive rate and 100% true-positive rate** at a cosine threshold of 0.99. In a separate speed benchmark (100 Dolly prompts, SmolLM2-135M-Instruct, Apple MPS, bf16, greedy, max 256 tokens), fingerprint capture adds **1.3% provider overhead** and verification takes **56.8 ms** per response — a single forward pass, **~5.5% of generation wall time**, roughly **18 $\times$ faster than re-executing inference**. The on-chain commitment is **256 bytes** regardless of response length. The entire system runs end-to-end on a local Solana test validator today.

1. Motivation

1.1 Two problems, one network

Today, if you want to run a modern language model you send a request to one of a handful of companies — OpenAI, Anthropic, Google, a GPU cloud vendor — and trust them to actually run the model you asked for. That trust is unverifiable. Nothing in the response proves which model was run. A provider could quietly route your request to a cheaper, worse model to save compute. They could truncate the generation early. They could return a cached response. You would not know.

Separately, there is an enormous and growing supply of GPUs that are *not* owned by those few companies — in independent data centers, in gaming rigs, in research labs, in corporate clusters that are idle at night. These machines are capable of running the same models the hyperscalers run, often at a lower total cost, but they have no good way to sell their capacity. The buyer has no way to trust them.

dInference is a network that connects the two sides. Anyone with a GPU can register as a provider and serve inference requests. Anyone can submit a request for an LLM completion and pay for it in SOL. And — the hard part — **anyone paying for a request can afterwards prove, cheaply and quickly, that the response really came from the model they asked for.**

If an inference provider cheats, they lose money. If they are honest, they get paid. The network does not need a central referee because the proof is in the response itself.

This pattern — an open set of operators contributing real-world physical capacity, coordinated and compensated through on-chain primitives — is known as a **decentralized physical infrastructure network, or DePIN**. Filecoin applies it to disk storage, Helium to wireless coverage, Render to GPU rendering. dInference applies it to LLM inference. In every case the appeal is the same: tap into underutilized hardware that lives outside the hyperscalers, pay operators per unit of real work delivered, and let open participation drive price down and quality up. The ingredient that was missing for inference — a way to tell whether a stranger actually ran the model they said they ran — is what the rest of this paper builds.

1.2 Why this is a hard problem

Making an open network for AI inference work requires solving one specific problem: *how do you tell whether a stranger on the internet actually ran the model they said they ran?* Three answers have been tried and none are satisfying.

Rely on reputation. Reviews, uptime scores, slow off-chain dispute processes. This is just trust in a different wrapper. It is slow, political, and does not help in adversarial conditions where the attacker only needs to cheat once.

Run the model yourself to check. Have N independent nodes run the same request and compare outputs. This works but multiplies cost by N ; it erases any pricing advantage over the centralized incumbent and does not scale.

Produce a cryptographic proof of the computation (zkML). Published zero-knowledge-proof systems for LLM inference are roughly 10^2 – $10^4 \times$ slower than running the model natively. A product that lives or dies on tokens-per-second cannot absorb that overhead on every request. The gap is closing year over year, but for the gigabyte-scale models that drive real demand it is still far from practical.

1.3 The dInference approach

dInference uses a fourth answer: **optimistic verification with a cheap, highly accurate check.** By default, the network trusts the provider — the response flows back to the user, the provider gets paid, and no verification happens. If the user suspects the response is wrong, they open a *challenge*. The challenge triggers a small committee of independent verifiers who run the response through the claimed model and check a tiny mathematical signature (called an **activation fingerprint**) that the provider was required to commit on-chain when they claimed the bounty. If the fingerprint matches, the provider is paid and the challenger loses their challenge bond. If it does not, the provider is slashed and the challenger is reimbursed.

Two properties make this cheap enough to work in practice:

- **Verification runs at native speed.** Checking a response takes one forward pass through the model — the same work the provider did once, done once more. In our benchmarks (§4.5) this is around 57 milliseconds for a few-hundred-token response on a small model. There is no cryptographic overhead.

- **The commitment is tiny.** The activation fingerprint is 64 floating-point numbers (256 bytes), regardless of how long the response is. The same 256 bytes verify a 10-token answer and a 10,000-token answer.

The rest of this paper describes how the fingerprint works (§4), how the on-chain program coordinates it (§7), how the economics keep providers honest (§5, §6), and how the end-to-end system is implemented and benchmarked today.

2. Protocol overview

Three actor roles:

- **User.** Posts a bounty for inference on a specific prompt against a specific model, receives the response, and is the only party who may open a challenge on their own bounty.
- **Provider.** Runs the model locally; races other providers to claim bounties for models they serve.
- **Verifier.** Stakes capital against a particular model; is drafted into verification committees when a challenge opens.

A single inference request flows through:

1. **Post.** User uploads the prompt to Walrus (content-addressed decentralized storage on Sui testnet), submits `post_bounty(nonce, prompt_txid, bounty_lamports)` on Solana, and escrows SOL in the global stake vault. Chain emits `BountyPosted`.
2. **Claim.** Every provider daemon subscribed to the model receives the event, fetches the prompt, runs inference, uploads the response, and submits `claim_bounty(response_txid, fingerprint)`. First valid transaction to land atomically flips the bounty status from `Open` to `Claimed`; losing transactions revert.
3. **Challenge window.** For a configurable number of slots, the user may call `challenge(bond)` with a bond proportional to the bounty. The on-chain program deterministically selects K verifiers from the model's pool using `hash(rotating_seed || bounty_pda || i) mod pool.len()` for $i = 0..K$, swap-removing each selected verifier so no operator sits on two simultaneous committees.
4. **Vote.** Each committee member fetches prompt and response, teacher-forces the pair through its local copy of the claimed model, recomputes the fingerprint, computes cosine similarity against the on-chain commitment, and submits `verifier_vote(verdict, fp_cos_scaled)`.
5. **Early finalize.** As soon as the running vote tally makes the outcome decidable (quorum of guilty reached, or honest majority locked), anyone may call `finalize_challenge`. Payouts, slashes, and pool re-insertions execute atomically.
6. **Optimistic settlement.** If no challenge opens within the window, anyone may call `finalize_payment` and the full bounty flows to the winning provider.

3. Optimistic rollup rationale

The verification system is a direct application of the optimistic-rollup pattern used by Ethereum L2s like Arbitrum and Optimism: *assume honest by default, backstop with cheap on-demand verification*. In those systems the cheap backstop is a fraud-proof game over the state transition. Here it is the activation-fingerprint check.

The design decision this pattern turns on is: *are challenges rare in equilibrium?* If they are, the amortized verification cost across all requests is near zero, and the system runs at native inference speed. If they are not, verification cost compounds and the system is worse than redundant execution. For dInference the answer is yes-rare: in a well-tuned economy, every challenge opened by a user has an expected outcome visible to the user before they file — if their response looks correct, there is no payoff in filing. Challenges only happen when a response looks wrong, and well-tuned providers rarely produce wrong responses because they lose stake when they do.

Three concrete properties the optimistic model buys dInference:

- **No work on the happy path.** The vast majority of requests settle after `finalize_payment` with zero verifier compute spent. The *existence* of the committee is enough deterrent; it rarely has to actually run.
- **Permissionless early finalize.** The moment the vote tally mathematically locks (quorum of guilty cast, or remaining votes can no longer reach quorum), anyone — not just the committee — may close the challenge. This drops typical dispute resolution from a full window to a few blocks.
- **Deterministic reproducibility of selection.** Because committee selection uses an on-chain rotating seed and public pool state, SDKs can predict committee membership before the on-chain transaction lands. This matters for off-chain daemon scheduling (verifiers can pre-fetch artifacts for challenges they will shortly be assigned).

4. The verification protocol

This is the central mechanism of the paper and deserves its own section. **Verification is a heuristic — not a cryptographic proof — but it is empirically very accurate.** The verifier recomputes the exact same vector the provider committed, over the model’s own forward pass, using the same seeded projection. In the honest case the two vectors agree up to bfloat16 numerical roundoff; in the cheating case they are near-orthogonal. The threshold used — cosine similarity > 0.99 — absorbs the bf16 roundoff. The main accuracy result lives in §11: a 500-prompt $\times$ 3-scenario cross-family evaluation (1,500 samples) in which the check separates honest and cheating runs with 0% false-positive rate and 100% true-positive rate. §4.5 reports only the timing benchmark.

4.1 Fingerprint construction

For a request with on-chain seed s and claimed model M of hidden dimension d , the protocol fixes a projection dimension k (dInference uses $k = 64$) and a seeded Gaussian projection matrix $P \in \mathbb{R}^{d \times k}$ derived deterministically from s . During generation, the provider captures the final-layer hidden state $h_i \in \mathbb{R}^d$ at the position that produced each generated token $i = 1 \dots n$. The fingerprint committed on chain is:

$$f = \sum_{i=1}^n h_i P \in \mathbb{R}^k$$

Three properties follow immediately:

- **Fixed size.** f is always k float32 values, regardless of n . On chain this is 256 bytes.
- **Model-specific trajectory.** Because h_i depends on the full generation history up to position i under model M , the sum f encodes the trajectory, not just the final state. A different model produces different h_i at every step; the projected sums are uncorrelated.

- **Seeded uniqueness.** P changes per request. A provider who memorized a prior f cannot reuse it; a verifier replaying with the wrong seed gets a random-looking vector.

4.2 Verifier recomputation

On challenge, each committee member fetches (prompt, response) from Walrus, reconstructs the full tokenized sequence [prompt_ids ++ response_ids], runs a single forward pass through the claimed model with hooks on the final RMSNorm to capture the per-position hidden state, selects the states at positions (prompt_len - 1) ... (prompt_len + n - 2) (the states that produced each response token under a causal LM), recomputes f' the same way, and checks:

$$\cos(f, f') > \tau \quad (\tau = 0.99)$$

The arithmetic is identical to the provider’s: same model, same projection matrix, same hidden states, same sum. In the honest case the two vectors differ only by bf16 numerical noise accumulated across different KV-cache paths (streaming decode on the provider side vs single-pass teacher-forcing on the verifier side). This noise is empirically small — $\cos > 0.999$ in nearly all honest runs.

4.3 Why the activation level

A natural question: why fingerprint hidden states, not output logits or sampled tokens? Three reasons:

- **Sampling-level checks are economically pointless.** A provider who swaps a different sampler (greedy for temperature, say) saves nearly zero compute — they still paid the full forward passes. Attacks that save real compute (serving a smaller model, serving no model) show up as hidden-state divergence long before they show up as token divergence.
- **Logit comparison is wasteful.** Logit vectors are much larger than k and must be computed per-step; comparing them would blow up both commitment size and verification work for no additional security.
- **Hidden states carry the model’s identity.** The hidden-state manifold is the model. A projection of that manifold under a seeded matrix is a cheap fingerprint of “which model produced these tokens on this input.” That is exactly the quantity the user cares about.

4.4 Why not zkML

Published zkML systems (EZKL, Modulus Labs, and related work) report proving costs on the order of 10^2 – $10^4 \times$ the native forward pass for small-to-medium models — large enough that a per-request DePIN cannot absorb the overhead on the hot path. The gap narrows every year, and for small models with structured proving pipelines it is already within a few orders of magnitude of native; for the GB-scale LLMs that drive real user demand it remains prohibitive. The fingerprint check instead runs at native speed and leans on *economic* finality rather than *cryptographic* finality: a cheater is caught, their stake is slashed, and the open supply side self-corrects.

4.5 Measured timing

Timing benchmarked on 100 prompts from the Databricks Dolly 15k instruction dataset (the cross-family *accuracy* evaluation at 500 prompts $\times$ 3 scenarios lives in §11). SmoLM2-135M-Instruct on Apple M-series MPS in bfloat16, greedy decoding, max_new_tokens = 256, same projection seed on every run:

Metric	Mean	Median	p95
Baseline generation (no capture)	1013.7 ms	1271.8 ms	1687.4 ms
Provider generation + fingerprint capture	1027.2 ms	1313.7 ms	1631.0 ms
Verifier (single teacher-forced forward pass + cosine)	56.8 ms	58.3 ms	103.2 ms

Derived:

- **Provider overhead: +1.3% total wall time, +0.8% per token.** The hook captures a single d -dimensional vector per generation step; the projection is done once, after generation, on the stacked tensor. Overhead sits in measurement noise.
- **Verifier wall time: 56.8 ms, which is 5.5% of provider wall time — a $\sim 18\times$ speedup over re-running inference.** This is the entire verification cost: one forward pass, no decoding, no sampling, no allocator churn across steps.
- **Fingerprint quality: mean $\cos = 0.99964$, min **0.99435**, max **0.99999**.** All 100 passed the $\tau = 0.99$ threshold.
- **On-chain commitment: 256 bytes ($64 \times \text{float32}$),** independent of response length.

The size claim is worth restating: **verifying an arbitrarily long response commits 64 floats on chain and costs one additional forward pass to check.** This is the headline result of the paper. zkML needs the model, the proof, and megabytes-to-gigabytes of SNARK witnesses; redundant execution needs K full copies of the inference; dInference needs 256 bytes and a cheap forward pass only when a challenge is actually opened.

4.6 Improvements still open

Three obvious optimizations we have not yet implemented:

- **Batched verification.** A verifier assigned to several concurrent committees could batch their forward passes. SmoLLM2-135M on an A100 would saturate under batching; grouping 8–16 challenges into a single pass drops per-challenge verification into the ones of milliseconds.
- **Raw token-id commitments.** Today the provider decodes the response to text (with special tokens preserved for the verifier’s benefit); the verifier re-tokenizes to reconstruct the sequence. BPE can re-split tokens differently at the prompt/response boundary, which in practice does not seem to lower $\cos$ below threshold but is theoretically fragile. Having the provider upload the raw generated token IDs removes this reconstruction step entirely.
- **Per-layer hooks for smaller commitments.** The current fingerprint uses the final layer only. A projection of several layers’ hidden states, or a selection chosen per-model, could tighten numerical margins and allow smaller k with equal discriminative power.

5. Provider competition and supply-side dynamics

The provider side is a first-come-first-served race. Every provider daemon runs the claimed model on local hardware; the first valid `claim_bounty` transaction to land wins the full bounty, and losers eat the compute they spent on a request they did not win. This single rule produces the supply-side dynamics the DePIN narrative depends on.

Hardware improvements convert directly into revenue. A provider on an H100 beats a provider on an A100 beats a provider on consumer silicon. Better accelerators, better compilation (TensorRT-LLM, vLLM, SGLang), better quantization that still passes the fingerprint check, better serving infrastructure — each one shortens claim latency, raises the share of races won, and raises revenue. There is no horizon past which spending more on hardware stops paying off, because the next request is always another race. In a well-populated network this drives providers toward the current frontier of inference efficiency without any central coordination.

Oversupply shrinks margins; undersupply attracts entry. When too many providers chase too few bounties, each wins less often, revenue per unit time falls, and the slowest drop out. When too few providers serve a model, bounties sit unclaimed, users raise prices, and new providers register. The network finds a per-model equilibrium where provider count $\times$ average tokens-per-second roughly tracks user demand.

Model specialization. A provider registers per-model via `register_provider(model_id, stake)` and need not serve every model. An operator with hardware or a serving stack tuned to one architecture (Qwen, say) can concentrate stake on that model and dominate its bounty flow without wasting resources on races they would not win.

5.1 Current daemon: FCFS without filtering

The current provider daemon (`off_chain_inference/ramp.py`) is deliberately simple: on every Bounty-Posted event for a model it serves, if it is not already busy, it picks up the bounty and runs the inference. No filtering by bounty amount, prompt length, estimated token count, or current queue state.

This is a proof-of-concept simplification, not a design limit. Natural improvements that follow directly:

- **Profit-aware selection.** Estimate tokens needed from prompt length, apply model-specific tokens/s, compute expected wall time, compare to bounty payout. Decline bounties where expected revenue per second falls below a configurable floor.
- **Queue scheduling.** Multiple concurrent generations if hardware permits; fairness across the daemon’s own queue; dynamic priority based on current pending-bounty depth.
- **Prompt-length cutoffs.** Decline unusually long prompts whose compute cost exceeds a threshold the operator has set.
- **Hardware-utilization awareness.** Switch between batched and single-request modes based on current GPU occupancy; pre-warm the KV cache for likely upcoming prompts.

These are off-chain scheduler decisions and change nothing about the on-chain protocol — a provider daemon can be arbitrarily sophisticated as long as it still submits a correct `claim_bounty` for the bounties it chooses to pursue.

6. Sybil resistance

Open participation plus random committee selection raises the obvious attack: can an adversary spin up N verifier accounts, dominate the pool, and skew committees in their favor?

The protocol defends against this with **capital at risk, not identity**. To join the verifier pool for a model, an operator must call `register_verifier(model_id, stake)` with `stake  $\geq$  min_verifier_stake` locked into the vault. Each additional sybil account requires another full stake. Scaling an attack to dominate a pool of size N with probability p of landing on any given committee requires $O(p \cdot N / (1 - p))$ sybil accounts, each paying full stake, all at risk of slashing whenever one of them votes against the eventual majority.

Three properties compound:

- **The stake is not a refundable bond.** It sits as collateral indefinitely and is slashed on dishonest votes. A sybil army that votes guilty on honest challenges to coerce a verdict is slashed on every honest challenge the network processes.
- **Slashes compound across sybils.** If a well-funded adversary stakes 100 sybils, every honest challenge in which one of them is drafted and votes dishonestly slashes that account; the attacker loses stake at a rate proportional to their pool share. The attack is not just capital-expensive; it is burn-rate-negative.
- **Committee selection is unpredictable before the challenge opens.** The rotating seed advances on a schedule that is faster than the challenge window. An attacker cannot pre-position sybils to be guaranteed selection on a specific bounty; they can only increase their probability of landing on *some* committee by increasing their pool share, which costs more stake, which costs more total slash exposure.

The equivalent provider-side defense works the same way: `register_provider` requires `stake ≥ min_provider_stake`, and `claim_bounty` additionally enforces `stake ≥ 2 \cdot (0.25 \cdot \text{bounty} + \text{verify\_pool})` so a half-stake slash covers both the challenger reward and the verify fees paid to the committee. A sybil provider cannot claim a bounty they are not collateralized to lose.

The protocol is not sybil-*proof* — enough capital at risk could still overwhelm a small pool — but the attack is explicitly economic, and the cost scales linearly with the fraction of the pool the attacker intends to control. The attacker’s loss on any single detected dishonest round is a configurable function of total capital at risk, which the admin can tune by changing `dishonesty_fee_lamports`.

7. On-chain program

The smart contract is written in Rust using the Anchor framework and compiled to BPF for Solana. Program ID: `EDX9NGUiJGtGtLDTHbRCqLrb6MbF7ES4eKuxumpfUyWq` (local test deployment).

7.1 Account model

Account	Purpose
<code>GlobalConfig</code>	Admin pubkey, stake minimums, bounty minimums, challenge/vote windows, fingerprint cosine threshold, committee size, quorum fraction.
<code>RotatingSeed</code>	Current 32-byte seed; rotates on a schedule via permissionless <code>rotate_seed</code> .
<code>StakeVault</code>	Single PDA holding all operator stakes and user bounty escrows.
<code>ModelInfo</code>	Per-model: id string, hidden dimension, allowed flag.
<code>VerifierPool</code>	Per-model: vector of currently-available verifier operator pubkeys.
<code>ProviderAccount</code>	Per-operator-per-model: stake, reserved stake, activation flag, unstake state.
<code>VerifierAccount</code>	Per-operator-per-model: stake, activation flag, unstake state.

Account	Purpose
Bounty	Per-request: user, prompt_txid, bounty_lamports, status, winner, timestamps, response commitment reference.
ResponseCommitment	Per-claim: response_txid, fingerprint vector ($64 \times \text{float32}$), submission slot, challenge deadline.
Challenge	Per-challenge: committee, verdicts, status, vote deadline.

7.2 Instructions

Admin: init_config, init_model.

Operator lifecycle: register_provider, register_verifier, request_unstake_*, claim_unstake_* (two-step unstake with cooldown $\geq$ challenge window).

Bounty lifecycle: post_bounty, claim_bounty, finalize_payment, reclaim_unclaimed_bounty.

Challenge: challenge, verifier_vote, finalize_challenge.

Housekeeping: rotate_seed.

7.3 Committee selection

Given a pool P and target committee size K:

```
for i in 0..K:
    idx_i = u64::from_le_bytes(hash(rotating_seed || bounty_pda || i)[0..8]) % P.len()
    committee.push(P.swap_remove(idx_i))
```

$O(K)$ hashes, $O(K)$ swap-removes. Deterministic — any SDK can replay the chain to predict committee membership. Swap-remove prevents simultaneous dual-committee assignment. Surviving (not-slashed) committee members are re-inserted after finalize_challenge.

7.4 Early-finalize decidability

Let n = committee size, g = guilty votes cast, h = honest votes cast, $r = n - g - h$ remaining, $q = \lfloor n \cdot \text{quorum_fraction} / 10,000 \rfloor$. Decidable iff:

- $g \geq q$ (slash the provider), or
- $g + r < q$ (acquit).

finalize_challenge is permissionless once decidable.

7.5 Stake invariants

Provider at claim time: $\text{stake} \geq 2 \cdot (0.25 \cdot \text{bounty} + \text{verify_pool})$. Guarantees a half-stake slash covers challenger reward ($1.25 \times$ bounty on success) plus verify fees. reserved_stake is incremented at claim, decremented at finalize, preventing double-booking across concurrent claims.

Verifier at register time: $\text{stake} \geq \text{dishonesty_fee}$ so one slash cannot underflow.

8. Off-chain components

Python 3.11, solders for transaction building, solana-py for RPC, transformers for model execution, httpx for Walrus I/O.

8.1 Fingerprint pipelines

Provider (`off_chain_inference/fingerprint_pipeline.py`) subclasses `transformers.Pipeline`. Preprocess applies the tokenizer’s chat template for type: `instruct` models (per-model config in `off_chain_inference/config/models.yaml`), raw tokenization for type: `base`. Forward registers a forward hook on `model.model.norm` (final RMSNorm) so one hidden-state vector is captured per generation step; generate runs with its normal kwargs (no `output_hidden_states`), the hook populates a Python list, and after generation we stack and project. The hook approach is critical — the obvious alternative (`output_hidden_states=True` on generate) materializes every layer at every step and runs 50%+ slower.

Verifier (`off_chain_verifier/fingerprint_pipeline.py`) mirrors the chat template handling, concatenates `prompt_ids ++ response_ids`, runs a single forward pass with the same final-norm hook, selects the response positions, recomputes the projection, computes cosine. Returns `{fp_cos, passed, truncated, reason}`.

Both pipelines decode / handle special tokens so the completion-rule check works: if `n_tokens < max_new_tokens` and the last token is not a declared stop token, the response is flagged `truncated_without_eos` and voted guilty. This catches providers who return early to save compute without a legitimate stop.

8.2 Daemons

Provider daemon (`off_chain_inference/ramp.py`) subscribes to Solana `logsSubscribe` filtered by program ID, parses `Program data:` lines against known Anchor event discriminators. On each `BountyPosted` for its model: Walrus fetch → inference → Walrus upload → `claim_bounty`. Single-slot busy flag; FCFS; no smart selection (see §5.1 for the natural extensions).

Verifier daemon (`off_chain_verifier/ramp.py`) subscribes to the same stream. On each `ChallengeOpened` whose committee includes its operator: fetch both blobs → fingerprint check → `verifier_vote` → opportunistically call `finalize_challenge` if the tally is decidable. A failed blob fetch auto-votes guilty — provider failed to post what they claimed.

8.3 User SDK

`DIInferenceClient.inference(prompt, model_id)` uploads prompt to Walrus, posts bounty, polls the response commitment PDA, fetches the response, strips special tokens, returns `(bounty_pda, text)`. `.challenge(bounty_pda, model_id)` posts bond and opens challenge.

9. Storage layer

Walrus (Sui testnet) was selected over Arweave after library-compatibility issues with `arweave-python-client` on locally-generated JWKs and after measuring 60-second indexing lag on Turbo-SDK / Irys. Walrus publishes return a content-addressed blob ID (sha256 of content, base64url) immediately; aggregator GETs resolve in 1–2 seconds with no indexing lag. Content-addressing lets the chain commit a blob ID before

the blob is retrievable and lets verifiers confirm independently that what they fetched is what the provider committed.

10. Attack surface

Attack	Caught by
Run a different model	Fingerprint check; $\cos \approx 0$.
Don't run the model at all	Fingerprint check; no valid hidden states.
Truncate response to save compute	Completion-rule check: last token must be a declared stop token if $n_tokens < max_new_tokens$.
Sampling tampering (greedy, temperature)	Not caught by design; attack saves no compute, is economically irrational.
Provider wins race, then stalls	Challenge window + <code>claim_bounty</code> reverts on re-attempt.
User spams fake challenges	Loses bond on every wrong outcome.
Verifier collusion to vote guilty	Minority voters slashed; constant verify fee removes directional incentive.
Verifier doesn't vote	Treated identically to a minority vote (slashed).
Two providers claim same bounty	Atomic status flip; second claim reverts.
Provider under-stakes relative to bounty	<code>claim_bounty</code> enforces $stake \geq 2(0.25 \cdot bounty + verify_pool)$.
Sybil pool stuffing	Capital-at-risk per sybil + per-round slash exposure (see §6).
MEV / pre-reveal of committee membership	Committee hash includes rotating seed, which rotates faster than challenge window.

11. Offline evaluation across model families

Evaluated on 500 prompts $\times$ 3 scenarios (1,500 samples total) on Modal A10G GPUs:

Scenario	Model actually served	Mean cos	Pass rate
Honest	Qwen2.5-1.5B-Instruct	1.0000	1.0000
Cheat (smaller same-family)	Qwen2.5-0.5B-Instruct	-0.0059	0.0000
Cheat (cross-family similar size)	SmolLM2-1.7B-Instruct	-0.0018	0.0000

At $\tau = 0.99$: FPR = 0, TPR = 1, accuracy = 1.

The live local-validator end-to-end runs reproduce the honest-path result: $\cos = 1.0000$ on challenged honest responses, no slashing, both provider and verifier stakes intact after acquittal.

12. Limitations

Economic parameters are placeholders. The live `GlobalConfig` currently has `verify_fee_lamports = 0`, `dishonesty_fee_lamports = 0`, `challenger_reward_bps = 0`, both time windows near U64 max. The

mechanism is wired through — reserved stake tracks claim-time escrow, slashing arithmetic preserves vault invariants, pool reinsertion is idempotent — but the knobs are untuned. Realistic values are required before any adversarial deployment.

Tokenization-boundary fragility. The verifier reconstructs the provider’s token sequence by concatenating `chat_template(prompt) + tokenize(response, add_special_tokens=False)`. BPE can re-merge tokens differently at the join; in practice this has not pushed cos below threshold on any tested model, but is theoretically unsound. The clean fix is for the provider to upload raw generated token IDs and have the verifier teacher-force directly on those.

Dtype determinism. `bfloat16` computations are not bitwise-identical across GPU architectures and driver versions. The 0.99 threshold has significant slack, but behavior at the limit (`fp16` vs `bf16`, consumer vs datacenter GPU) has not been characterized systematically.

Walrus is testnet. Mainnet Walrus is live but pipeline currently targets Sui testnet endpoints; mainnet pricing / retention / SLA have not been evaluated.

No prompt privacy. Prompts and responses are cleartext on public decentralized storage. For sensitive use cases, client-side encryption with keys derived from the user’s keypair is the obvious extension but is unimplemented.

Only Solana. Protocol is not chain-specific in principle; committee selection and slashing port to any environment with cheap deterministic hashing and native transfers. No other chain has been attempted.

13. References and reproducibility

Smart contract (`programs/dinference/`), off-chain pipelines (`off_chain_inference/`, `off_chain_verifier/`), chain client (`chain/`), user SDK (`sdk/`), end-to-end orchestration (`scripts/local_e2e.sh`), and the fingerprint benchmark used in §4.5 (`scripts/bench_fingerprint.py`) together constitute roughly 4,000 lines of Rust and Python.

The cross-family evaluation harness (`scripts/experiment_cross_family.py`) runs on Modal and produces the TPR / FPR numbers in §11.

Author’s note

This is a personal proof of concept by **Dhruva Chayapathy**, a student at the **Georgia Institute of Technology**. Claude (Anthropic) was used as a coding and design-review assistant throughout the implementation of the smart contract, off-chain daemons, SDK, benchmark harness, and this document. All design decisions and the final shape of the system are the author’s; Claude contributed code, patterns, and critique against the author’s direction.

The system has been exercised end-to-end on a local Solana validator; it has not been audited, deployed to a public cluster under adversarial load, or had its economic parameters tuned for any production use.